UNIVERSITÉ LAVAL

Solutionnaire du TP3

À votre tour

0. Importer et vérifier

```
comptes <- read_tsv("/home/public/stat/airway_counts.tsv") |>
  column_to_rownames("gene") |>
  as.matrix()
infos <- read_tsv("/home/public/stat/airway_sample_info.tsv") |>
  select(Run, cell, dex)
```

```
dim(comptes)
#> [1] 63677     8
infos
#> # A tibble: 8 × 3
#>   Run        cell    dex
#>   <chr>      <chr>   <chr>
#> 1 SRR1039508 N61311  untrt
#> 2 SRR1039509 N61311  trt
#> 3 SRR1039512 N052611 untrt
#> 4 SRR1039513 N052611 trt
#> 5 SRR1039516 N080611 untrt
#> 6 SRR1039517 N080611 trt
#> 7 SRR1039520 N061011 untrt
#> 8 SRR1039521 N061011 trt
identical(colnames(comptes), infos$Run)
#> [1] TRUE
```

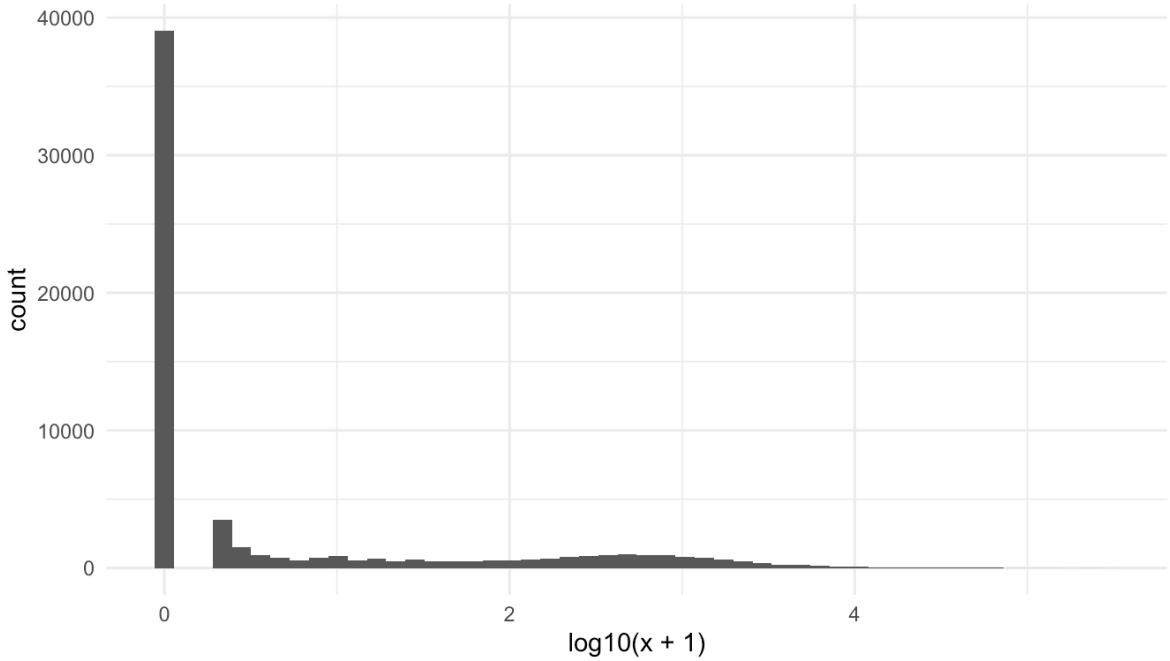
63 677 gènes, 8 échantillons, dans le même ordre.

1. Explorer les comptes

```
colSums(comptes)
#> SRR1039508 SRR1039509 SRR1039512 SRR1039513 SRR1039516 SRR1039517 SRR1039520
#>   20637971   18809481   25348649   15163415   24448408   30818215   19126151
#> SRR1039521
#>   21164133
sum(rowSums(comptes) == 0)
```

```
#> [1] 30208

ggplot(tibble(x = comptes[, 1]), aes(log10(x + 1))) +
  geom_histogram(bins = 50)
```



`count()` est écrit `dplyr::count()` : DESeq2 charge le package `matrixStats`, qui a aussi une fonction `count()` et masque celle de `dplyr` (voir les messages au chargement).

De 15 à 31 millions de lectures selon l'échantillon : la **profondeur** varie, d'où la normalisation de DESeq2. Près de la moitié des gènes n'ont aucune lecture : ils ne sont pas exprimés dans ce tissu.

2. Construire l'objet DESeq2

```
infos <- infos |>
  mutate(dex = factor(dex, levels = c("untrt", "trt")),
         cell = factor(cell))
levels(infos$dex)
#> [1] "untrt" "trt"

dds <- DESeqDataSetFromMatrix(countData = comptes,
                              colData = column_to_rownames(infos, "Run"),
                              design = ~ cell + dex)
dds <- dds[rowSums(counts(dds)) >= 10, ]
nrow(dds)
#> [1] 22369
```

`converting counts to integer mode` : un message, pas une erreur (les comptes ont été lus comme des nombres décimaux). Il reste 22369 gènes.

3. Le test

```
dds <- DESeq(dds)
res <- results(dds)
res
#> log2 fold change (MLE): dex trt vs untrt
#> Wald test p-value: dex trt vs untrt
#> DataFrame with 22369 rows and 6 columns
#>                  baseMean log2FoldChange     lfcSE       stat      pvalue
#>                 <numeric>      <numeric> <numeric>  <numeric>   <numeric>
#> ENSG00000000003  708.5979     -0.3812271 0.1007023  -3.785684 1.53286e-04
#> ENSG00000000419  520.2963      0.2068404 0.1121077   1.845014 6.50354e-02
#> ENSG00000000457  237.1621      0.0379543 0.1428231   0.265744 7.90437e-01
#> ENSG00000000460   57.9324     -0.0885309 0.2849344  -0.310706 7.56024e-01
#> ENSG00000000971 5817.3108      0.4264246 0.0888056   4.801775 1.57266e-06
#> ...                   ...            ...       ...        ...         ...
#> ENSG00000273483   2.68955       0.849208  1.253365  0.6775423    0.498062
#> ENSG00000273485   1.28646      -0.123613  1.588251 -0.0778298    0.937963
#> ENSG00000273486  15.45244      -0.150429  0.482098 -0.3120292    0.755018
#> ENSG00000273487   8.16327       1.045639  0.693057  1.5087349    0.131367
#> ENSG00000273488   8.58437       0.108946  0.632300  0.1723006    0.863201
#>                       padj
#>                  <numeric>
#> ENSG00000000003 1.28920e-03
#> ENSG00000000419 1.94930e-01
#> ENSG00000000457 9.09900e-01
#> ENSG00000000460 8.92994e-01
#> ENSG00000000971 2.06391e-05
#> ...                     ...
#> ENSG00000273483          NA
#> ENSG00000273485          NA
#> ENSG00000273486    0.892519
#> ENSG00000273487    0.323298
#> ENSG00000273488    0.943415
summary(res)
#>
#> out of 22369 with nonzero total read count
#> adjusted p-value < 0.1
#> LFC > 0 (up)       : 2610, 12%
#> LFC < 0 (down)     : 2224, 9.9%
#> outliers [1]       : 0, 0%
#> low counts [2]     : 4337, 19%
#> (mean count < 5)
#> [1] see 'cooksCutoff' argument of ?results
#> [2] see 'independentFiltering' argument of ?results
```

Première ligne : `log2 fold change (MLE): dex trt vs untrt`, le **traité par rapport au non traité**. Un log2FoldChange positif signifie que le gène est **plus exprimé sous dexaméthasone**.

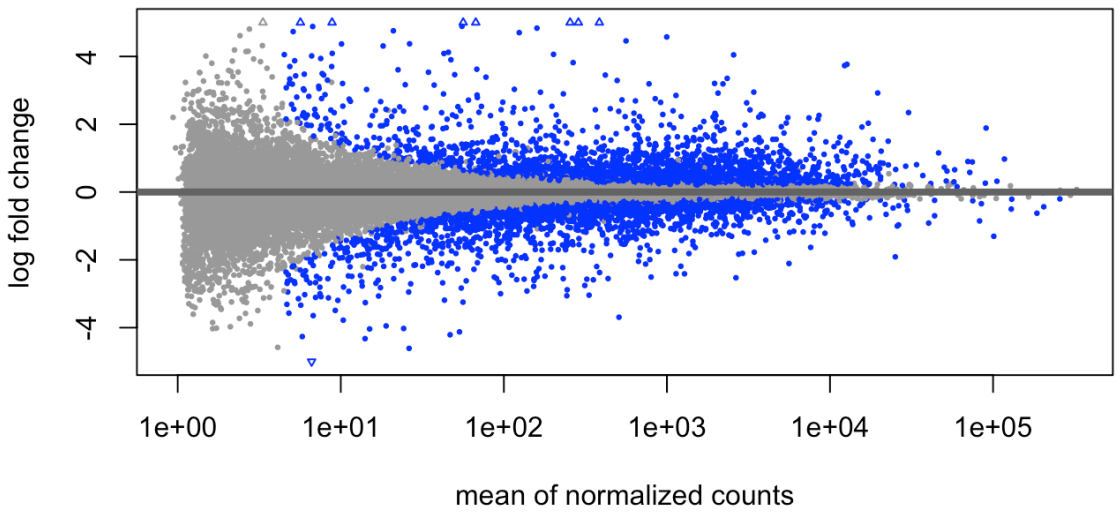
4. Compter les gènes DE

```
sum(res$padj < 0.05, na.rm = TRUE)
#> [1] 4000
sum(res$padj < 0.05 & abs(res$log2FoldChange) > 1, na.rm = TRUE)
#> [1] 1018

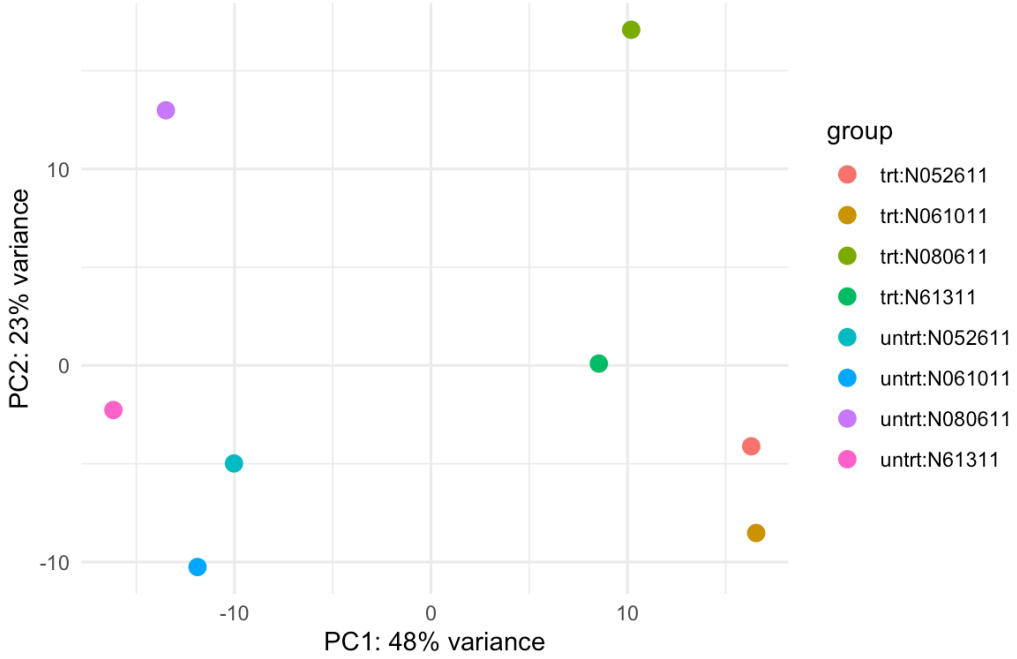
res_tbl <- as.data.frame(res) |>
  rownames_to_column("gene") |>
  filter(padj < 0.05, abs(log2FoldChange) > 1)
dplyr::count(res_tbl, sens = if_else(log2FoldChange > 0, "induit", "réprimé"))
#>      sens   n
#> 1  induit 531
#> 2 réprimé 487
```

5. MA-plot et ACP

```
plotMA(res, ylim = c(-5, 5))
```




```
vsd <- vst(dds, blind = FALSE)
plotPCA(vsd, intgroup = c("dex", "cell"))
```

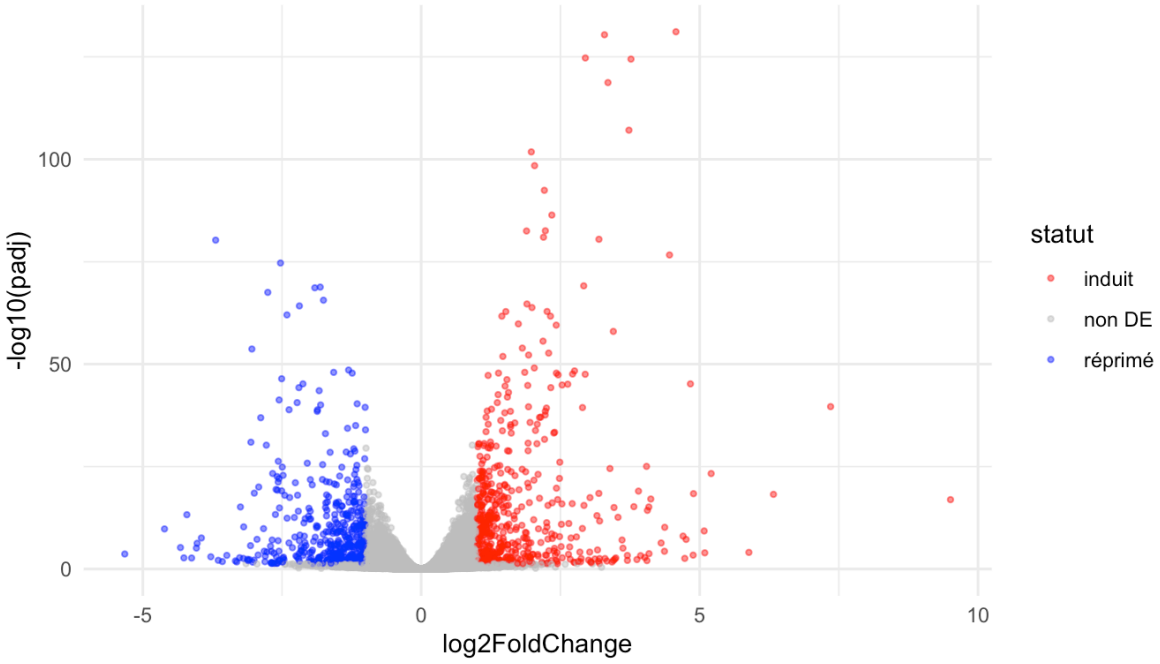


PC1 sépare les traités des non-traités : c'est la principale source de variation. PC2 sépare les lignées : un effet « donneur » réel, qui justifie de l'inclure dans le plan (`~ cell + dex`).

6. *Volcano plot*, top 10 et export

```
res_df <- as.data.frame(res) |>
  rownames_to_column("gene") |>
  filter(!is.na(padj)) |>
  mutate(statut = case_when(padj < 0.05 & log2FoldChange > 1  ~ "induit",
                            padj < 0.05 & log2FoldChange < -1 ~ "réprimé",
                            .default = "non DE"))

ggplot(res_df, aes(log2FoldChange, -log10(padj), colour = statut)) +
  geom_point(alpha = 0.5, size = 0.8) +
  scale_colour_manual(values = c(induit = "red", `réprimé` = "blue", `non DE` = "grey"))
```




```
res_df |>
  filter(padj < 0.05) |>
  arrange(desc(log2FoldChange)) |>
  select(gene, baseMean, log2FoldChange, padj) |>
  head(10)
#>                gene   baseMean log2FoldChange         padj
#> 1  ENSG00000179593  67.242812       9.505998 1.180357e-17
#> 2  ENSG00000109906 385.071328       7.352659 2.321950e-40
#> 3  ENSG00000250978  56.318557       6.326641 6.033984e-19
#> 4  ENSG00000132518   5.654707       5.884991 8.859193e-05
#> 5  ENSG00000127954 286.388753       5.207195 5.064476e-24
#> 6  ENSG00000249364   8.839183       5.092763 1.129372e-04
#> 7  ENSG00000171819 254.882599       5.081610 5.178592e-10
#> 8  ENSG00000100033  55.363920       4.889871 4.090612e-19
#> 9  ENSG00000260802   6.723144       4.883578 4.110417e-04
#> 10 ENSG00000168309 159.530276       4.834999 6.531708e-46
```

```
res_df |> filter(statut != "non DE") |> write_csv("genes_DE_airway.csv")
```

Parmi les gènes les plus induits, plusieurs ont un `baseMean` faible : un grand log2FoldChange calculé sur peu de lectures est moins fiable. `lfcShrink()` (vignette de DESeq2) corrige cet effet pour classer les gènes.

Et si… ?

1. `sum()` sans `na.rm`

```r
sum(res$padj < 0.05)
#> [1] NA
sum(is.na(res$padj))
#> [1] 4337
```

`NA` : au moins une `padj` est manquante, donc la somme aussi. 4337 gènes n'ont pas de `padj` : DESeq2 ne teste pas les gènes trop faiblement exprimés (**filtrage indépendant** : ils n'ont aucune chance d'être significatifs, et les tester alourdirait la correction pour les autres) ni ceux qui ont une valeur aberrante. `NA` ne veut pas dire « non DE » : cela veut dire « non testé ».

2. `pvalue` au lieu de `padj`

```r
sum(res$pvalue < 0.05, na.rm = TRUE)
#> [1] 5690
sum(!is.na(res$pvalue)) * 0.05
#> [1] 1118.45
```

5690 gènes au lieu de 4000. Si aucun gène ne réagissait, on en attendrait environ **1118** sous le seuil de 5 % par le seul hasard (5 % des gènes testés). La `padj` (FDR) limite à environ 5 % la proportion de faux positifs **parmi les gènes retenus**.

3. Ignorer l'appariement

```r
dds_dex <- DESeqDataSetFromMatrix(countData = comptes,
                                  colData = column_to_rownames(infos, "Run"),
                                  design = ~ dex)
dds_dex <- DESeq(dds_dex[rowSums(counts(dds_dex)) >= 10, ], quiet = TRUE)
sum(results(dds_dex)$padj < 0.05, na.rm = TRUE)
#> [1] 2694
```

Moins de gènes DE : sans `cell`, la variabilité **entre donneurs** reste dans le « bruit » du modèle et masque une partie de l'effet du traitement. C'est le même phénomène que le test indépendant appliqué aux données appariées de `sleep` (TP1).

4. Des échantillons dans le désordre

```r
DESeqDataSetFromMatrix(countData = comptes[, c(2, 1, 3:8)],
                       colData = column_to_rownames(infos, "Run"),
                       design = ~ cell + dex)
#> Error in DESeqDataSetFromMatrix(countData = comptes[, c(2, 1, 3:8)], colData = column_
#>    SRR1039508,SRR1039509,SRR1039512,SRR1039513,SRR1039516,SRR1039517,SRR1039520,SRR103
#>   are not in the same order as the colnames of the countData:
#>    SRR1039509,SRR1039508,SRR1039512,SRR1039513,SRR1039516,SRR1039517,SRR1039520,SRR103
```

Une **erreur**, heureusement : DESeq2 compare les noms des colonnes de `countData` à ceux des lignes de `colData` et refuse de continuer. Lisez le message : il donne les deux ordres. Sans noms (une matrice sans `colnames`), aucune vérification ne serait possible : d'où l'intérêt de garder les identifiants.

5. Sans fixer le niveau de référence

```
infos_alpha <- infos |> mutate(dex = factor(as.character(dex)))   # ordre alphabétique
levels(infos_alpha$dex)
#> [1] "trt"   "untrt"
dds_alpha <- DESeqDataSetFromMatrix(countData = comptes,
                                    colData = column_to_rownames(infos_alpha, "Run"),
                                    design = ~ cell + dex)
dds_alpha <- DESeq(dds_alpha[rowSums(counts(dds_alpha)) >= 10, ], quiet = TRUE)
res_alpha <- results(dds_alpha)
mcols(res_alpha)$description[2]
#> [1] "log2 fold change (MLE): dex untrt vs trt"
c(trt_vs_untrt = res["ENSG00000152583", "log2FoldChange"],
  untrt_vs_trt = res_alpha["ENSG00000152583", "log2FoldChange"])
#> trt_vs_untrt untrt_vs_trt
#>     4.574967    -4.574966
```

`trt` vient avant `untrt` dans l'alphabet : le **traité** devient la référence, et la première ligne affiche `dex untrt vs trt`. Tous les log2FoldChange changent de **signe** : *SPARCL1*, fortement induit par la dexaméthasone, apparaîtrait « réprimé ». Les p-values, elles, sont identiques : rien dans les étoiles ne trahit l'erreur. Seule la **lecture de l'en-tête** la révèle.